

`componentWillMount` or the constructor as we saw before, the `render` method renders the component in the browser. The next method is `componentDidUpdate`, and it is executed when the new component has been updated in the DOM. This method is used to make sure that the various third parties we use in the code are also updated and have reloaded themselves.

Similarly, there are some methods that get called when the state gets updated; so you may notice that the methods under the `states` column in Figure 1 are a subset of the updating process of props, and they behave in the same way as we discussed in the case of props. The only difference is that they deal with the state instead of props.

Now the component is in the last phase of its life cycle, i.e., 'unmounting'. `componentWillUnmount` is the last method of the life cycle and is executed just before the component gets removed from the DOM. We can do all the cleanups related to the component here.

We can see that there are too many methods in the React life cycle and so much to remember as to their functioning. This becomes more difficult as our codebase increases in complexity. After getting many queries and providing explanations for where one method is better than the other, React has come up with Hooks.

What are React Hooks?

Hooks are a new addition in React 16.8, and they let you use state and other React features without writing a class. If you're already using a class in your current React application, there is no need to change it. There are currently no plans to remove classes from React, and hooks don't replace your knowledge of React. So whatever you know about React is intact; Hooks will just change the way you create components.

Why use React Hooks?

Hooks are functions that let you hook into React state and life cycle features from function components. You can easily add a new component using hooks to your existing React app, and it will not at all interfere with your current functionality. So why do we change the way we currently code for React, and why should we go for React Hooks?

Currently, it's very difficult to use stateful logic between components. Hooks make our task of state management easier. With hooks, you can extract stateful logic from a component so that it can be tested independently and be reused. It also allows you to reuse stateful logic without changing your component hierarchy, i.e., the component tree remains the same. This makes it easy to share hooks among many components. Also, hooks let you split one component into smaller functions based upon pieces that are related, rather than forcing a split based on the life cycle methods; so this makes them simple and more readable. Moreover, classes can be confusing, especially if you have a large codebase. If you have experience with React, you know

you need to bind the event handlers, the code is lengthy, and the distinction between function and class components in React (and deciding when to use each) adds to the mayhem. As React Hooks totally avoids the classes, this mess can be avoided too.

Transitioning from class to function

Let's see how we can transform from a class component to a function component using the most basic programming example 'hello world'.

```
class MyApp extends React.Component {
  render() {
    return <h1>Hello, {props.name}</h1>;
  }
}
```

This is a class component in React. As we saw before, we extend the React component and have a render method that returns the elements we need to show on the browser:

```
function MyApp(props) {
  return <h1>Hello, {props.name}</h1>
}
```

Now, moving to the function component, the first thing we can see is that we totally avoid this keyword and all the inconsistencies around it, which I feel is a victory in itself, and we avoid the render method too! The component returns 'hello world' with the 'h1' tag. Of course, this is a simple example; so understanding the level to which function components can simplify your life is a bit difficult to visualise here. But when you have massive classes doing many things, the simplicity of moving towards functions will be more apparent and hooks will let you do that seamlessly.

Categorising hooks

Let's see what React Hooks actually are. React has three basic hooks, which we will be using the most. There are a few additional hooks that aren't common but have their own use, and it doesn't hurt to know about them. So we will go through each and see how they work.

useState

```
import React, { useState } from 'react';

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
    </div>
  );
}
```